



DEPARTMENT OF ENGINEERING MATHEMATICS

Design and Implementation of a Query-Driven Hybrid Video Game Recommendation System with Hardware Requirement Enrichment

Vikrant Nitin Deshmukh

A dissertation submitted to the University of Bristol in accordance with the requirements of the degree
of Master of Science in the Faculty of Engineering.

Friday 4th September, 2026

Supervisor: Dr. Karina Nurlybayeva

Declaration

This dissertation is submitted to the University of Bristol in accordance with the requirements of the degree of MSc in the Faculty of Engineering. It has not been submitted for any other degree or diploma of any examining body. Except where specifically acknowledged, it is all the work of the Author.

Vikrant Nitin Deshmukh, Friday 4th September, 2026

Contents

1	Introduction	1
1.1	Project Context and Motivation	1
1.2	Problem Statement	1
1.3	Aim and Objectives	2
1.4	Research Questions	2
1.5	Project Contributions	3
1.6	Thesis Structure	3
2	Background and Related Work	4
2.1	Recommender Systems and Video game Recommendation	4
2.2	Content-based & Item-to-item game recommendation	4
2.3	Textual and Semantic Recommendation	5
2.4	Graph-Based Recommendation	6
2.5	Hybrid Recommendation and Rank Fusion	6
2.6	Evolution of Recommender Systems	7
2.7	Context and Hardware Requirements	8
2.8	Research Gap	8
3	System Design and Execution	9
3.1	Dataset Selection and Preparation	9
3.2	Popularity Baseline	10
3.3	Weighted Metadata Recommender	11
3.4	Exploratory Rich-Text TF-IDF Recommender	12
3.5	Transformer-Based Semantic Recommender	13
3.6	Graph-Based Recommender	14
3.7	Hybrid Recommendation Framework	16
3.8	Hardware Requirement Enrichment Layer	18
4	Critical Evaluation	19
4.1	Evaluation Design	19
4.2	Metrics Evaluation	21
4.3	Comparison of Recommendation Approaches	22
4.4	Metadata-Graph Overlap Analysis	24
4.5	RRF Weight Sensitivity Analysis	25

4.6	Qualitative Case Studies	26
5	Conclusion	28
5.1	Summary of Contributions	28
5.2	Achievement of Project Objectives	28
5.3	Limitations	29
5.4	Future Work	30
5.5	Final Conclusion	30

List of Figures

1	Overall Hybrid Recommendation System Architecture	9
2	Bipartite Graph Structure of the Graph Recommender	14
3	Weighted Reciprocal Rank Fusion Framework	17
4	Quantitative Comparison of Recommendation Approaches in terms of Effectiveness and Runtime	23
5	Metadata-Graph Recommendation Overlap	25
6	RRF Weight Sensitivity Analysis	26

List of Tables

1	Hyper-parameters for Graph Recommenders	15
2	Formal Query Games used for Evaluation	20
3	Relevance Scale used for Query–Candidate Assessment	21
4	Overall Quantitative Comparison of the Recommendation Models	23
5	Metadata–Graph Recommendation Overlap at Different Cut-offs	25
6	RRF Weight Sensitivity Results	26
7	Selected Qualitative Recommendation Examples	27

Ethics Statement

This project fits within the scope of the University of Bristol Data Science blanket ethics application 15208, as reviewed by my supervisor, Dr Karina Nurlybayeva.

Abstract

The rapidly expanding number of entries on a digital game’s catalogue makes finding a new game an increasing difficult task for consumers. With thousands of games available across many platforms there are numerous factors that affect whether or not a consumer will purchase a particular title. These include but are not limited to the genre(s) of the game, gameplay mechanics, and minimum/maximum system specifications required by the game. This research presents the results of designing and developing a hybrid video game recommender system which incorporates data from a Steam catalogue consisting of 89,618 games.

This research evaluates multiple ways to measure the similarities between two games. First, this research develops a metadata based recommender that utilizes genres, steam tags and gameplay modes as attributes. Second, it develops a transformer-based semantic recommender that uses natural language processing techniques to determine the similarities between descriptive game text. Third, it develops a graph based recommender that uses relationships between games and other entities in the Steam catalogue (genres, tags, developers, etc.) to determine similarities. Additionally, this research also includes the use of a non-personalized popularity baseline and a rich-text TF-IDF model for comparison purposes. Finally, these recommendations are then fused together using weighted reciprocal rank fusion (RRF) so that heterogenous ranked output can be used to create a single hybrid recommendation list without having to directly calibrate their respective similarity scores.

These recommendations were tested against each other using the same collection of formal queries and metrics of relevance, ranking quality, intra-list diversity and runtime. The hybrid recommender was able to achieve the highest precision at ten items ($\text{Precision@10} = .990$), whereas the graph recommender had the highest normalized discounted cumulative gain at ten items ($\text{NDCG@10} = 0.9638$). The semantic recommender was able to produce the highest diversity at ten items (0.819). In addition to testing the individual components of the hybrid recommender, additional analyses were performed to test the amount of overlap between the metadata and graph models’ recommendations as well as the sensitivity of the hybrid ranking to varying RRF weight configurations.

In addition to providing a hybrid recommender system for query driven video game recommendation, this research extends its potential by incorporating a recommendation-driven hardware requirement enrichment pipeline. The final objective to be fulfilled as defined in the enrichment area was that of an implementation of a pipeline for a Hardware Requirements Enrichment based on recommendations. For this purpose, minimum and recommended system specs to run each game were queried via the Steam Store API, saved locally, processed into structured hardware fields and merged with the recommended games. Aspects related to user-PC hardware compatibility evaluation and hardware-based re-ranking are currently outside the focus of the enrichment objectives realized and should be considered for future development.

The overall system allows for a comparative framework of query-driven video game recommendation systems as well as demonstrates how hardware requirement information may be added as a post-recommendation enrichment layer. Full user specific hardware compatibility assessment and hardware-based re-ranking are kept as areas for future research.

Supporting Technologies

The implementation and evaluation of the proposed recommendation system used the following principal technologies:

Technology	Role in the Project
Python	Primary programming language used for data preparation, recommender implementation, evaluation and hardware-requirement processing.
Pandas and NumPy [15, 11]	Used for catalogue manipulation, preprocessing, numerical operations and management of recommendation outputs.
SciPy [26]	Used for sparse matrix representations required by high-dimensional metadata and graph-based feature spaces.
Scikit-learn [17]	Used for TF-IDF vectorisation, vector normalisation, cosine-based similarity calculations and nearest-neighbour retrieval.
Sentence Transformers [19, 22]	Used with the pre-trained <code>all-mpnet-base-v2</code> model to generate 768-dimensional semantic embeddings from game descriptions.
CUDA [16]	Used for GPU-accelerated generation of semantic embeddings when compatible hardware was available.
Google Colab	Primary notebook-based environment used for experimentation, implementation and evaluation.
Steam Store API	Used to dynamically retrieve minimum and recommended PC system requirements for games returned by Hybrid recommender.
CSV Storage	Used to retain intermediate recommendation outputs, evaluation data, runtime measurements and cached hardware-requirement information.

Semantic embeddings and retrieved hardware requirements were stored for reuse, reducing unnecessary recomputation and repeated API requests during subsequent stages of the recommendation pipeline.

Code and Data Availability. The implementation notebooks, processed datasets and evaluation artefacts associated with this dissertation are available in the accompanying GitHub repository:

[GitHub Repository](#)

Acknowledgments

I would like to express my sincere gratitude to my dissertation supervisor, Dr Karina Nurlybayeva, for her guidance, feedback and support throughout the development of this project. Her advice was particularly valuable in refining the scope of the dissertation, evaluating the technical approaches considered, and maintaining a clear direction throughout the different stages of the work.

I would also like to thank the University of Bristol and the teaching staff of the MSc Data Science programme for providing the academic foundation, resources and learning environment that supported both this project and my postgraduate studies.

Finally, I am deeply grateful to my family and friends for their continued encouragement, patience and support throughout my MSc. Their reassurance and presence were especially valuable during the demanding final stages of completing the project and dissertation.

Chapter 1

Introduction

1.1 Project Context and Motivation

Digital game distribution platforms offer an extensive library of games with respect to numerous genres; types of player experiences (play style); subject matter; mechanisms used in the game; and system capabilities needed to run each game. This increase in diversity offers users greater options for their preferred gaming experience. However, increased options result in a larger amount of time required to find new games which share some of the same properties as previously identified favorite games. A user could have a favorite game but require an exhaustive search of a large number of games within the platform's catalogue to determine if there are other titles that share characteristics similar to those found within their previous favorite.

Recommender Systems present a viable approach to limit the number of games from which a user needs to select when looking for new titles. Recommenders create lists of items that are relevant to a specific query or reference item. Video games can be compared based on several sources of data. Sources include genres, Steam tags, various features related to gameplay, written descriptions, and the relationships between developers and publishers. Various recommender approaches use differing aspects of these data.

For example, structured metadata can express certain characteristics about games explicitly. Semantics can help compare similarities between the meanings contained within written descriptions. Graphs can describe relationships between games and all other catalogue objects.

In addition to providing recommendations relevant to discovering new games, another important consideration regarding practical game discovery is the hardware requirements imposed by PC games. Requirements relate to processor ability, graphics processing unit, RAM, hard drive capacity, and operating-system compatibility. The hardware requirements are typically presented as semi-structured text and are often unavailable in formats which allow for computerized analysis.

Therefore, this research will investigate a query-driven recommender system where the user identifies a game title and then receives a ranked list of related games. Following identification of related games, the recommendation phase is enhanced with a hardware requirement enrichment layer designed to extract and format the minimum and recommended PC specifications associated with the returned games. An objective of this research is to integrate game-related similarities with useful hardware specification information without combining the two processes into one methodology.

1.2 Problem Statement

This dissertation addresses two related challenges in query-driven video game recommendations:

1. **Capturing multiple forms of game similarity.** A single recommendation approach may not represent all relevant relationships between games. Structured metadata can capture explicit similarities such as genres and tags, semantic models can identify similarities in descriptive meaning, and graph-based representations can model structural relationships involving developers, publishers

and other catalogue entities. Relying on only one of these signals may therefore overlook useful relationships between games.

2. **Integrating hardware requirement information with recommendation outputs.** PC game system requirements are often provided separately from recommendation information and commonly appear as semi-structured textual descriptions. Processor and graphics-card requirements in particular may contain heterogeneous model names and free-form descriptions, while memory and storage requirements may be represented using different units or formats. Consequently, useful hardware requirement information must be retrieved, cleaned and structured before it can be incorporated reliably into the recommendation pipeline.

The project therefore investigates a hybrid recommendation framework that combines complementary recommendation signals and subsequently extends the resulting recommendations with hardware requirement information. This provides a foundation for generating recommendations that are both relevant and, in a later compatibility stage, practically suitable for the user's system.

1.3 Aim and Objectives

The overall aim of this dissertation is to design, implement and evaluate a query-driven hybrid video game recommendation system that combines multiple game-similarity representations and enriches the resulting recommendations with structured PC hardware requirement information.

To achieve this aim, the following objectives were defined:

1. Prepare and preprocess a large-scale Steam games dataset suitable for query-driven recommendation and comparative evaluation.
2. Develop multiple game recommendation approaches, including weighted metadata, lexical text, transformer-based semantic and graph-based methods, together with a non-personalised popularity baseline.
3. Develop a hybrid recommendation framework that combines the principal Metadata, Semantic and Graph recommenders using Weighted Reciprocal Rank Fusion.
4. Formally evaluate the individual and Hybrid recommendation approaches using relevance, ranking quality, diversity, recommendation-overlap and computational-performance measures.
5. Develop a recommendation-driven hardware requirement enrichment pipeline that retrieves, caches and structures minimum and recommended PC system requirements for games returned by the Hybrid recommender.

1.4 Research Questions

The dissertation is guided by the following research questions:

1. **RQ1:** How do weighted Metadata, Transformer-based Semantic and Graph-based recommendation approaches differ in terms of recommendation relevance, ranking quality, diversity and behaviour?
2. **RQ2:** How does the Hybrid ranking approach compare with its constituent recommenders in terms of relevance, diversity and ranking performance metrics?
3. **RQ3:** How can PC hardware requirement information be retrieved, structured and integrated as a post-recommendation enrichment layer within a query-driven video game recommendation pipeline?

These questions are addressed through the comparative evaluation of the individual recommendation approaches, the Weighted Reciprocal Rank Fusion framework and the hardware-requirement enrichment process described in the following chapters.

1.5 Project Contributions

The main contributions of this dissertation are as follows:

1. **Design and comparative implementation of multiple game representations.** The project develops and compares Metadata, lexical TF-IDF, Transformer-based Semantic and Graph-based approaches for query-driven game recommendation using a common Steam catalogue.
2. **Implementation of a rank-fusion Hybrid recommendation framework.** The principal Metadata, Semantic and Graph recommenders are integrated using Weighted Reciprocal Rank Fusion, allowing heterogeneous ranked outputs to be combined without direct calibration of their raw similarity scores.
3. **A structured comparative evaluation framework.** The recommendation approaches are evaluated under a common set of formal queries using relevance, ranking quality, intra-list diversity and runtime measures. Additional analyses examine Metadata–Graph recommendation overlap and the sensitivity of the Hybrid model to alternative RRF weight configurations.
4. **A recommendation-driven hardware requirement enrichment pipeline.** PC system requirements are retrieved dynamically for recommended games, cached to reduce repeated requests, and transformed from semi-structured Steam descriptions into more structured processor, graphics, memory, storage and operating-system fields.

1.6 Thesis Structure

The remainder of this thesis is organised as follows. **Chapter 2** reviews relevant literature on content-based, textual, semantic, graph-based and recommender-system evaluation approaches. **Chapter 3** describes dataset preparation and the implementation of the individual recommenders, the Weighted RRF Hybrid framework and the Hardware Requirement Enrichment pipeline. **Chapter 4** formally evaluates the principal Metadata, Semantic, Graph and Hybrid recommenders using relevance, ranking quality, diversity and runtime measures, together with recommendation-overlap, RRF weight-sensitivity and qualitative analyses. The hardware component is considered as an implemented enrichment pipeline rather than a formally evaluated compatibility recommender. Finally, **Chapter 5** summarises the findings, addresses the research questions and project objectives, discusses limitations and outlines directions for future work.

Chapter 2

Background and Related Work

2.1 Recommender Systems and Video game Recommendation

Recommender systems are information-filtering systems designed to help users navigate large collections of items by identifying alternatives that are likely to be relevant to their interests or current information needs. Their importance has increased alongside the growth of digital platforms, where users are frequently exposed to more products, media items and services than can be examined manually. Recommendation research has consequently developed a range of approaches for reducing this information overload and supporting item discovery [13, 28].

Traditional recommender systems are commonly divided into collaborative filtering, content-based filtering and hybrid approaches. Collaborative filtering derives recommendations primarily from patterns of historical user-item interaction, whereas content-based methods rely on descriptive attributes of users or items. Hybrid systems combine multiple information sources or recommendation strategies in order to compensate for limitations associated with any individual approach [13]. More recent recommender systems have also incorporated richer behavioral, contextual and representational information, reflecting the increasing variety of data available on modern digital platforms.

The need for effective recommendation is particularly evident in the video-game domain. Digital distribution platforms such as Steam contain large and continually expanding catalogues, in which games differ across genres, gameplay modes, developers, publishers, textual descriptions, platform support and patterns of player engagement. Cheuque et al. [6] identify this catalogue diversity and the resulting difficulty of discovering suitable games as an important motivation for recommender systems on online gaming platforms. More recent work similarly argues that the growth of the game market has increased the need for specialised recommendation approaches capable of supporting discovery across large and heterogeneous catalogues [14].

Prior research on game recommendation has explored several recommendation paradigms, including collaborative filtering, content-based approaches, neural models and graph-based methods. Cheuque et al. compared an implicit-feedback ALS baseline with Factorization Machines, Deep Neural Networks and DeepFM using Steam data, while also evaluating novelty and diversity in addition to ranking accuracy [6]. Their work illustrates that game recommendation is not only a problem of predicting user preference, but also of exploiting the rich catalogue and interaction information available within gaming platforms. The following sections therefore examine content-based, semantic, graph-based and hybrid approaches in greater detail.

2.2 Content-based & Item-to-item game recommendation

Content-based recommendation uses descriptive information about the individual items (e.g. movies, books, etc.) for recommending items to the end-user; it does not rely solely on aggregated user behavior. In terms of video-games, this would mean utilizing the various game attributes (i.e., e.g., Genres, Tags, Reviews, Textual Descriptions), and other gaming related attributes. Viljanen et al. [25] demonstrated how useful game attributes could be for the development of content-based interaction models for games.

Specifically, they concentrated upon addressing the cold start problem. For example, Collaborative Filtering is often unable to generate effective recommendations for either: New games/players since the only available interaction history was generated by the game/player itself. The authors found that content based models were generally better at making predictions involving new games/new players/new both than collaborative filtering. Therefore, there appears to exist another benefit to incorporating content into recommendations. That is, recommendations can continue to occur even when there are limited/historical interaction records available.

However, the choice of features will impact what a content-based system views as being "similar". Ryan et al. [20] argued that standard generic classifications used by industry (genres) represent only a small portion of all relationships between games. For instance, two games that belong to completely different genres could also share many additional associations including common developers, fictional worlds, characters, style and so forth. As such, to identify these broader relationships Ryan et al. introduced the concept of *relatedness*. More specifically, relatedness identifies similarities across multiple types of association, whereas traditional systems limit similarity to an explicitly defined genre hierarchy.

To implement this conceptualization of relatedness Ryan et al. employed Latent Semantic Analysis (LSA) on the Wikipedia articles pertaining to the games. LSA represents words within the articles (describing each game) in a multi-dimensional vector space. Candidate titles were then evaluated using Cosine Similarity to rank them according to their similarity to the input title(s).

Ryan et al.'s resultant game recommender demonstrated a precision value of .70 versus .57 (chance); thereby demonstrating the feasibility of using similarity to games that are already well-known or liked as a viable means for creating recommendations [20]. Nevertheless, due to its limited number of participants (10 users) the authors acknowledged limitations regarding the generalizability of their findings. Overall, these results indicate that descriptive information about individual games can support recommendations independent of having a large amount of interaction histories. Furthermore, meaningful similarities among games may extend beyond simple categorizations. Thusly, it seems reasonable to utilize more sophisticated semantic/textual representations for capturing these similarities.

2.3 Textual and Semantic Recommendation

The research has shown an evolution of latent textual representations toward dense transformer-based embeddings. Also it is noted that the two types of relatedness, i.e., semantic similarity and behavioral preferences, are fundamentally distinct. Therefore, the use of semantic similarity in recommender systems will likely need to be supplemented with other relevant data.

Ryan et al. [20] studied game-relatedness via the application of Latent Semantic Analysis to the content of Wikipedia articles describing video games; they were able to find connections among games that would have otherwise been ignored because they fell outside of traditional genre categories. Although their study found that text-based similarity metrics can offer a valuable source of recommendation signals, they recognized that Latent Semantic Analysis models rely upon statistical regularities inherent to the base corpus used to train them.

A more contemporary method to derive semantic similarity from text is represented by the use of transformer-based architectures. Specifically, Reimers and Gurevych [19] developed Sentence-BERT (SBERT) — an architecture based upon the usage of siamese and triplet networks to compute a single vector embedding per input sentence. These sentence embeddings are then capable of being compared efficiently through the use of cosine similarity. In addition, unlike the process required when utilizing a standard cross-encoder BERT model where each pair of sentences must be embedded together prior to comparison, Sentence-BERT enables independent computation of embeddings for each input sentence; this independence makes Sentence-BERT particularly well-suited for large scale retrieval applications. For example, the authors showed that comparing 10,000 sentences, which would have taken roughly 65 hours under a pairwise BERT inference regime, took just about five seconds using SBERT embeddings.

In a later study, Gao et al. [10] identified the importance of the training objective in determining the quality of semantic embeddings. They proposed SimCSE — a method for generating semantic embeddings that utilizes contrastive learning for the generation of those embeddings. Additionally, Vančura et al. [24] identified that general purpose transformers for processing sentences cannot automatically recognize behavioral patterns contained in user-item interaction data. As a result, they proposed their beeFormer

framework for training sentence-transformers in conjunction with both textual side information and interaction data. This allowed them to report improved performance in cold start, zero-shot and temporal recommendation scenarios relative to baseline models trained solely on textual side information.

2.4 Graph-Based Recommendation

Graph-based recommender models are based upon both items/users/auxiliary features represented as nodes connected together through edges and therefore allow the exploitation of structural properties of the data that cannot easily be captured using independent feature vector representations [2]. Examples of the various types of graph models used in recommender systems are: User-Item Bipartite Graphs Heterogeneous Graph Models Knowledge Graph Models Item-Attribute Graph Models [2]. These graph structures are particularly helpful when there are many ways in which relationships impact the relevance of items. However, the type of graph model selected along with how the model will propagate this information (or what the propagation mechanism will look like), will have substantial implications on the behavior of the recommendations produced by these models [2].

Recently, researchers have made extensive use of Graph Neural Networks (GNNs) to learn node representations that take into account information aggregated from neighboring nodes [2]. Anand and Maurya [2] detail GNN-based recommender systems in terms of the graph structure used, the network architecture of the model, and learned graph embeddings; they further discuss some limitations of GNN-based models related to complexity and interpretability. One such prominent example of a GNN-based recommender system is the Knowledge Graph Attention Network (KGAT), proposed by Wang et al. [27]. KGAT uses a higher-order attention mechanism to assign different weights to each neighbor relationship between a user and/or item and a knowledge graph entity [27]. Using an attention mechanism to weigh neighbor relationships, allows KGAT to leverage higher order connections between users/items/knowledge-graph entities than other methods; it was shown that improving connection orders improves recommendation performance. However, once an optimal number of propagation layers is determined, increasing that number does not produce significant improvements in recommendation quality [27].

In addition to general application of graph modeling techniques for video-game recommendation, several recent studies have focused specifically on graph modeling for video-game recommendation. Specifically, Li et al. [14] developed CPGRec – which is designed to operate on a user-item bipartite graph where neighborhood information for both users and games includes additional attributes about the games themselves (i.e., categories/popularity) [14]. Furthermore, since CPGRec was developed to address the trade-offs involved in generating accurate recommendations vs. generating diverse recommendations, authors provided explicit mechanisms within their framework to ensure increased exposure to "long-tail" games [14]. Recently, Zhang et al. [29] examined whether leveraging play-time guided multimodal random walk could provide a means for traversing relationships between games while supporting cross-category discovery.

As stated earlier, graph-based approaches provide a versatile method for capturing and utilizing heterogeneous relationships between objects [2]. Additionally, however, increasingly complex neural networks (propagation/attention etc.) create greater requirements for training data as well as computational resources [2]. This indicates that structural characteristics of the graph itself can be a strong indicator for recommending relevant items regardless of whether trained GNN-based architectures are utilized.

2.5 Hybrid Recommendation and Rank Fusion

In hybrid recommender systems, multiple recommendation approaches are used together to leverage the respective strengths of individual recommendations and to mitigate the weaknesses of individual recommendations. An early classification of hybrid recommender systems was proposed by Burke [4], who classified hybridization into several categories such as weighted, switching, mixed, cascade, feature-combination, feature-augmentation and meta-level approaches. The differences in these hybrid approaches arise primarily in how the individual recommendations interact during either feature generation or output integration.

A recent systematic literature review conducted by Çano and Morisio [5] demonstrated the ongoing interest in hybrid recommendation. They reported that collaborative filtering was most often integrated into other recommendation techniques using weighted hybrids. Data sparsity and cold start were identified

as two major challenges that remain. This demonstrates why there is a desire for hybrid systems – each recommendation approach will identify some type of evidence; therefore, when they are used together it can provide a flexible alternative to identifying all possible evidence types through a single model or through interacting signals.

Rank aggregation is another mechanism for aggregating recommendations. When multiple recommenders generate separate ranked lists of items, rank aggregation provides a means of integrating the results. A method for performing rank aggregation of recommendations was developed by Cormack et al. [7]. Reciprocal Rank Fusion (RRF), which is an unsupervised technique for combining rankings from multiple information retrieval systems, uses the reciprocal of the position of each item within each system’s ranking as a contribution value. Higher-positioned items receive greater contributions. In addition to providing a method for combining rankings without having to compare the values associated with each item within the source rankings, RRF represents a method for combining rankings that does not require knowledge of the relative magnitude of the source rankings’ values.

More recently, the application of rank aggregation to recommender systems has been investigated further. Bałchanowski and Boryczka [3] have compared the performance of various rank aggregation techniques across the output of multiple recommendation algorithms. According to their study, rank aggregation can be beneficial for improving the overall quality of recommendations produced by individual recommendation algorithms. However, the effectiveness of individual techniques depends on specific characteristics of the recommendation scenario.

Therefore, previous studies indicate that hybridization and rank aggregation represent practical frameworks for fusing diverse recommendation evidence. Furthermore, due to the fact that constituent recommenders typically produce individually ordered outputs whose underlying scale(s) are typically non-comparable, rank-based fusion is especially well-suited for this purpose.

2.6 Evolution of Recommender Systems

Although recommender systems can be evaluated across multiple dimensions, no single metric provides a complete picture of recommendation quality. Zangerle and Bauer [28] therefore emphasise the importance of considering several complementary evaluation perspectives, including recommendation accuracy, ranking quality, diversity, novelty and coverage. They further note that optimising a system for one objective may negatively affect performance on other desirable properties. Consequently, evaluating recommender systems using multiple metrics provides a more balanced basis for comparing alternative recommendation approaches.

Precision measures the proportion of recommended items that are actually relevant for top-ranked item lists; however, ranking-based metrics such as NDCG [28] add an additional layer by rewarding those who place highly relevant items closer together near the top of their lists. A different type of desirable quality for a recommendation list is diversity. Instead of measuring whether or not the items in your list are similar to each other (intra-list diversity), we estimate intra-list diversity using the degree of dissimilarity between pairs of recommended items. This allows us to evaluate a system not only on whether or not it recommends relevant items, but also whether or not the alternative options provided are diverse [28].

Given the complexity of recommending games in particular, it is not surprising that researchers have identified that the same principles apply to this area as well. For example, Cheuque et al. [6] used two types of metrics to evaluate the Steam recommender – ranking accuracy and diversity/novelty – to illustrate that recommending games does not simply depend upon ranking accuracy. More recently, researchers have investigated this trade-off between relevance and diversity more directly. Li et al. [14], for example, described an “accuracy-diversity dilemma” where an increase in one of the two areas often leads to a decrease in the other. To address this challenge, they developed CPGRec to find a middle ground between the competing goals of relevance and diversity. Similarly, Zhang et al. [29] also looked into how to improve both accuracy and diversity, as well as promote cross-category discovery in recommending games.

In summary, collectively, this body of literature demonstrates that instead of trying to identify the “best” model out of all possible models from a single performance metric, it would be better to evaluate your recommender system using several complementary dimensions.

2.7 Context and Hardware Requirements

Recommendation relevance may depend not only on user preferences or item similarity, but also on the context in which a recommendation is consumed. Adomavicius et al. [1] describe context-aware recommender systems as approaches that incorporate additional situational information when generating or adapting recommendations. Device capability can therefore be considered one practical form of contextual information. The information about what users are able to do using their devices represent another form of contextual information. In fact, Ullah et al. [23], have provided a method for generating a recommendation based upon both the user's interests and the ability of the user's current access device to perform tasks relative to those recommended. Therefore, the usefulness of a recommendation depends on both being able to identify relevant items, as well as having sufficient capability to perform the required task.

In terms of PC gaming, there are several different types of requirements that games will impose; e.g., processor speed, amount of RAM, amount of disk space available etc. Prasetyadi and Mahfudin [18] built an expert system to determine the minimum computer configuration needed to run certain games given their requirement to play the game at specific settings. The study illustrated that there is a direct relationship between selecting a game and ensuring your computer has enough hardware capability to run that game.

Additionally, hardware related information typically needs to be processed prior to it being usable in computational applications. De Luisa et al. [8] collected game data from Steam (including game requirements) as un-processed text and identified the storage requirements from the text. The authors normalized the values to create a useable format for further research. As indicated above, extracting and converting hardware-related information is a typical process when creating computational models that utilize such information.

2.8 Research Gap

Overall, the literature review demonstrates that video-game recommendations have been addressed through Content-Based (CB), Semantic, Graph-Based and Hybrid methods; current research recognizes the need to achieve balance between Relevance, Ranking Quality, and Diversity. Although previous studies generally focused solely on Recommendation Quality, Contextual or Hardware Information was frequently examined outside of the recommendation process.

As demonstrated through prior work, Device Capability can affect the utility of Recommended Content, and System Requirements for PC Games are required to be extracted and normalized prior to use computationally. Nonetheless, throughout the Literature reviewed in this Study, there has been little consideration regarding the combination of Heterogeneous Game-Similarity Signals via Rank Fusion and subsequent enrichment of the resultant Recommendations with Structured PC Hardware Requirements.

This Dissertation addresses this gap by exploring a Query-Driven Hybrid Recommendation Framework which combines Metadata, Semantic and Graph-Based Similarity Signals and Enriches the Final Ranked Recommendations with Structured Hardware Requirement Information. The hardware component will serve as an additional layer added after the Recommendations are Generated rather than a Compatibility Filter tailored to each User's Personalized Device Profile; it provides a base for Future Extensions that will incorporate compatibility considerations.

Chapter 3

System Design and Execution

This chapter describes the overall design and implementation of the proposed query-driven Hybrid recommendation system. The complete workflow is illustrated in Figure 1, beginning with Steam catalogue preparation, followed by the individual recommendation models, Hybrid rank fusion, and the subsequent hardware requirement enrichment stage.

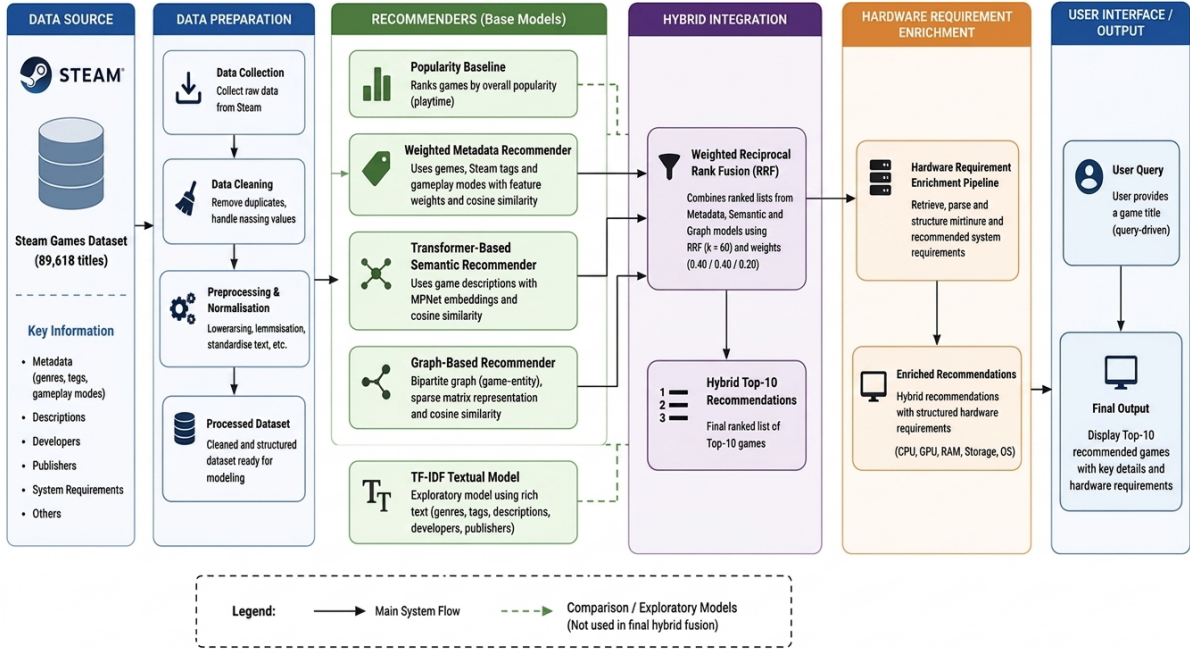


Figure 1: Overall Hybrid Recommendation System Architecture

3.1 Dataset Selection and Preparation

3.1.1 Source of the Data Set and the Data Selection

The project utilizes the publicly available steam game Dataset 2025, which is hosted on Kaggle [9]. The data set utilized in this project contains **89,618** game entries and has **47** data attributes. The 47 data attributes include title, Genres, categories, user defined tags, developers, publishers, textual descriptions, review statistics and engagements measures. Because the data set provided a mix of structured, textual, Semantic, graph based, and popularity oriented representations that were used at different times in this project, this combination supported all of these representations.

The steam application identifier (`appid`) was kept as the primary key to connect records between the

recommendation models developed in this project and those that were obtained via the external retrieval of PC hardware requirements. Even though the source data set had previously been processed prior to being used in this project, there still needed to be some additional project specific auditing and preprocessing of the data to create an appropriate modeling data set.

3.1.2 Data Audit and Attribute Selection

An initial audit was performed to examine the overall structure of the data set, identify any missing values in the data set and identify if there were any duplicated records within the data set. It was determined that there were no duplicate rows nor duplicate (`appid`) values. Out of the original 47 attributes used in the source data set, **30** were selected based upon how they fit into the overall recommendation pipeline.

Genres, categories and tags were used to support structured content modeling. About the game and short description were used by the textual and Semantic models. Developer and publisher information were used to support the creation of graphs. Recommendations, reviews, playtime and peak concurrent users were retained for use in popularity analysis. Any missing or unavailable values were addressed during preprocessing instead of discarding large amounts of catalog records.

3.1.3 Cleaning Data and Preparing Features

Release dates were changed to date-time format and a release year attribute was created. Values of -1 that are typically referred to as "sentinel" values when it comes to review related data that indicated that information was unavailable were replaced with missing values. After making this change, approximately 40.64% of the total review fields and 92.54% of recent review fields were missing values, so recent review information was not used as one of the main features for recommendations.

All textual fields were normalized by replacing missing values with empty strings, decoding HTML entities, removing all HTML tags and URLs, converting punctuation and non-alphanumeric characters to spaces, normalizing whitespace and converting text to lowercase. The original game title was saved separately for display purposes.

In addition to the original title field, Two derived textual representations were also created. `Metadata_text` combined cleaned Genres, categories and tags. `Combined_text` added additional fields including title and descriptive information to `metadata_text`.

Variables for engagement such as number of recommended items, positive and negative reviews, play time and peak concurrent users were converted to numeric form and transformed using `log1p` where appropriate to reduce the influence of extreme values on the model.

3.1.4 Final Data Set and Limitations

After completing preprocessing and creating features for the final data set, the final data set consisted of 89,618 games and 50 attributes and was saved to a file called `games_clean_v1.csv`. The final data set was used in all four notebooks including Metadata, rich-text tf-idf, Semantic and graph-based recommendation notebooks.

There still remained some missing descriptive information in the catalogue. There were 197 records without detailed description information. There were 220 records without about the game information. There were 120 records without short description information. All games that had at least one piece of missing description information were kept to preserve catalogue coverage. However, the quality of all text-based representations may be reduced by missing text. Validation also identified 85 games with empty `metadata_text` and Two games with empty `combined_text`.

3.2 Popularity Baseline

A non-personalised popularity baseline was developed as a catalogue-level reference against which the query-driven recommenders could be contextualised. Unlike the later models, it produced the **same ranked list** regardless of the query game. Four engagement variables were used: Steam recommendations,

positive reviews, negative reviews and peak concurrent users. Because these variables were highly right-skewed, $\log_{1p}$ transformations were applied to reduce the influence of extreme values.

The selected popularity score for title i is given as follows:

$$P_i = \log(1 + R_i) + \log(1 + Pos_i) - \log(1 + Neg_i) + \log(1 + C_i) \quad (3.1)$$

Here, R_i represents the quantity of Steam recommendations for title i , Pos_i denotes the positive reviews and Neg_i denotes the negative reviews, and C_i is the peak concurrent-user count.

This formula can be seen as a quality-based popularity metric rather than strictly measuring player interaction. Recommendations and peak concurrent-user numbers reflect how many players have been exposed to each title, providing a general sense of which titles have engaged the largest population. In addition to these popularity measures, the incorporation of positive and negative reviews adds a qualitative measure of each title’s reception. Although negative reviews will penalize a title when they exist, simply having large amounts of player interaction does not mean that those interactions are favorable. For instance, a widely played title may gain massive exposure through player interaction but simultaneously receive a lot of negative feedback. Subtracting out the negative review metric therefore diminishes the chance that the most prominent titles within a catalog dominate the ranking due purely to their size.

To assess whether or not incorporating review information significantly alters the ranking behavior, a sensitivity test was performed using an alternative engagement-only formulation:

$$P_i^{engagement} = \log(1 + R_i) + \log(1 + C_i) \quad (3.2)$$

In terms of the Top-20 rankings generated from both formulations, 14 titles existed in common, yielding a Jaccard similarity of roughly **0.54**. This illustrates that incorporating review information had a significant impact upon how titles are ordered. It should be noted however that since no user-level relevance “ground truth” existed for use with the popularity baseline, this comparison was not employed to demonstrate that one formulation is better than another based on empirical results. Rather, it serves as a sensitivity assessment supporting the notion that the chosen formulation could be viewed as a practical and quality adjusted catalog level benchmark.

3.3 Weighted Metadata Recommender

A query-driven content-based recommender system was designed to find other games that have a metadata profile that is very similar to the metadata profile of a game chosen by a user based upon a specific query title. The metadata recommender system produced a different ranked list for each unique query game compared to the popularity baseline, which produces a same ranked list throughout the entire catalogue for all users.

3.3.1 Metadata Feature Selection and Processing

Each group (Genres, Tags and Gameplay modes) was processed separately to allow for independent control of the contribution each made toward similarity. The metadata labels used in this analysis were parsed and normalized to eliminate differences in formatting that may affect how well a particular game’s features are represented across games.

Categories from Steam were further divided into those related to gaming and those related to the platform on which games can be played. Categories associated with the way players interacted with a game (such as single-player, multi-player, co-op or PVP), were included; categories related to the ability of the platform to store progress online (such as Steam Cloud), reward player achievement (achievements), allow family members to share access to content (Family Sharing), and/or use a controller (controller support), were excluded as they do little to indicate similarity in gameplay between different titles. Each group of features were then separately encoded as a sparse multi-label matrix.

3.3.2 TF IDF-Weighted Feature Representation

To prevent the excessive impact of overly common metadata values, an **Inverse Document Frequency (IDF)** based weighting scheme on each Genre, Tag, and Gameplay Mode Matrix were created.

The IDF weight for a given feature j was defined by the expression:

$$\text{IDF}_j = \log \left(\frac{N}{1 + df_j} \right) + 1 \quad (3.3)$$

where N is the total number of Games and df_j is the number of Games that contain feature j .

We then normalized each of our feature matrices using L2-Normalisation. Each feature matrix was then L2-normalised to place game vectors on a comparable scale before combination.

3.3.3 Feature Weighting

The three metadata groups were assigned field-specific weights:

$$w_{\text{tags}} = 0.60, \quad w_{\text{genres}} = 0.25, \quad w_{\text{play}} = 0.15.$$

These values were treated as **model-design assumptions** rather than parameters learned from labelled relevance data. Tags received the largest weight because they provide comparatively fine-grained information about gameplay mechanics, themes and atmosphere. Genres provide broader categorical information, while gameplay modes contribute complementary information about interaction style.

Each independently normalised matrix was multiplied by the square root of its assigned weight before concatenation. This preserves the intended relative contribution of each feature group when cosine similarity is later calculated. The combined metadata matrix was subsequently L2-normalised.

3.3.4 Candidate Generation and Similarity

In order to calculate how similar two games are in terms of their features, Cosine similarity on the final weighted metadata space was applied. The Brute Force Nearest Neighbor model was placed that uses Cosine Distance to find all nearest neighbors in L2 Normalized Sparse Representation. All metadata records from which there is no usable Genre, Tag or Gameplay-Mode data available were excluded from Metadata Based Retrieval.

The Catalogue Record associated with a Query Title was determined, and then a Pool of Nearest Metadata Neighbors were retrieved. The query game itself was removed, and cosine distance d was converted to similarity using:

$$S_{\text{metadata}} = 1 - d \quad (3.4)$$

Metadata Candidates were ranked based on unrounded Metadata Similarity Score. In cases where the number of candidates matches exactly (i.e., same Rank) Steam Application Identifier was used as a Deterministic Tie-Breaker. Similarity Scores were Rounded Only After Final Rankings Were Created.

For Interpretability purposes, Separate Genre, Tag and Gameplay-Mode Similarities were stored in addition to the Features Shared Between the Query and Recommended Games.

3.4 Exploratory Rich-Text TF-IDF Recommender

This structured metadata model was supplemented by developing a TF-IDF recommender based on descriptions [21] to see if Steam game descriptions' lexical information could provide another similarity signal. The `about_the_game` field, once cleaned, served as the primary source of text, and `short_description` was used as a fallback when needed. To avoid duplicating similar content, the two fields weren't combined.

Descriptions with fewer than 30 words were excluded due to insufficient text for comparison. This resulted in 87,568 eligible games from the 89,618-game catalogue, representing about 97.71% coverage.

TF-IDF vectors were created from eligible descriptions using uni-grams and bi-grams. English stop words were removed; Unicode accents were normalized, and vocabulary was capped at 30,000 features. Terms that appeared in fewer than three documents or more than 95% of them were excluded, and sublinear term-frequency scaling and L2 normalization were applied to reduce highly repeated terms' influence. The resulting sparse matrix had 87,568 game descriptions and 30,000 textual features.

Cosine distance and a brute-force nearest-neighbour model were used to find textually similar games, and similarity was calculated as $1 - d$. During development, top-50 candidate lists were generated to compare with other recommendation approaches.

This model was exploratory and **not included** in the final hybrid architecture. TF-IDF effectively captures explicit lexical overlaps but struggles with semantically similar games described differently. This approach provided a useful comparison between structured metadata similarity and the later semantic recommender based on transformers.

3.5 Transformer-Based Semantic Recommender

This recommender approach use transformers to find semantic connections between the games that may not be identified by similarities between metadata texts. It doesn't look at shared words like the TF-IDF model, but represents text as numbers to get the context. The semantic part is separated from genres, tags, and popularity, so it can add something new to the hybrid recommender later. This way, it can provide a different signal.

3.5.1 Text Representation and Embedding Generation

The `about_the_game` field which was cleaned in data preparation step, was the primary textual input, with `short_description` used only where the primary description was unavailable. Descriptions with fewer than 30 words were excluded, because extremely short descriptions were considered unlikely to provide sufficient contextual information for meaningful semantic representation. Due to which, **87,568** of the 89,618 catalogue games were eligible for semantic modelling. Input text was capped at 250 words to control length and cut excessive promotional content.

The pre-trained `sentence-transformers/all-mpnet-base-v2` model was used to generate semantic representations [19, 22]. It was used without fine-tuning and each eligible description was converted into a dense 768-dimensional embedding. The transformer sequence length was set to 384 tokens. No task-specific fine-tuning was performed because labeled game-pair relevance data were not available.

The maximum sequence length of 384 tokens was treated as an implementation setting rather than a tuned hyperparameter. Batch sizes of 64 on GPU and 16 on CPU were selected for computational efficiency and did not affect the semantic ranking formulation.

Embeddings were L2-normalised as they were generated and stored for reuse, rather than recomputing them for each query. That resulting embedding matrix therefore had dimensions (87,568 times 768). GPU acceleration was used for initial embedding generation when available. The implementation used CUDA on a Tesla T4 GPU for encoding, with a batch size of 64, and 16 for CPU fallback.

3.5.2 Semantic Similarity and Candidate Generation

For a selected query game, its pre-computed embedding was retrieved and compared with the embeddings of all other eligible games. Since the embeddings were L2-normalised, cosine similarity could be calculated directly using the vector dot product:

$$S_{\text{semantic}}(q, i) = \mathbf{q}^T \mathbf{x}_i \quad (3.5)$$

where $\mathbf{q}$ represents the query embedding and $\mathbf{x}_i$ represents the embedding of candidate game i .

To reduce unnecessary computation during recommendation retrieval, the query game’s own similarity score was excluded. Candidate indices with the highest similarity scores were first selected using a partial-partition operation and were then ordered by semantic similarity. This avoided the need to fully sort the complete catalogue for every query.

The recommendation function returned the Steam application identifier, game title, semantic similarity score and semantic rank. Additional catalogue information was retained for later interpretation and evaluation but did not influence the semantic ranking. For integration with the hybrid recommender, the top 50 unique semantic candidates were generated for each query game, with `appid` retained as the common identifier for combining outputs from the metadata and graph-based models.

3.6 Graph-Based Recommender

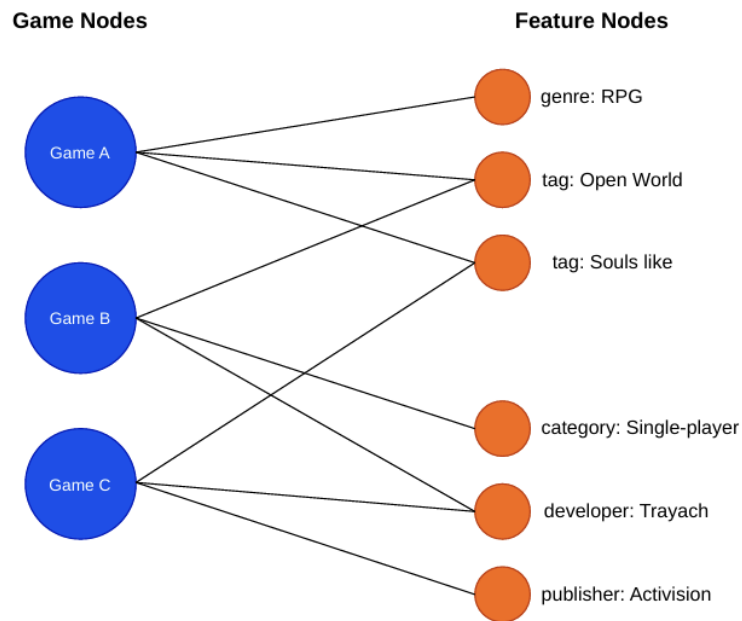


Figure 2: Bipartite Graph Structure of the Graph Recommender

A graph-based recommender was created in order to show relationships between games and catalogue elements all at once. The catalogue was shown as a bipartite graph (two partitions) where game nodes resided on one side and feature/entity nodes existed on the other. Additionally, five different feature types were included for representation; these being genre, category, Steam tags, developer and publisher. This would allow for the model to show both gameplay related features and organizational relationships of which may have been under-represented with the metadata recommender.

The approach does not use a learned GNN, message passing or node embeddings. Instead, the bipartite graph is represented as a weighted sparse incidence matrix, with game similarity calculated from weighted first-order shared relationships.

Figure 2 illustrates the basic bipartite structure used in the Graph recommender, where game nodes are connected to feature nodes representing genres, tags, categories, developers and publishers.

3.6.1 Graph Construction and pre-processing

The bipartite graph consisted of five relations; genres, categories, steam tags, developers and publishers. The first group of nodes of the bipartite graph contained the games, the second group of nodes contained the catalogue attributes. An edge existed between a node representing a game and a node representing a catalogue attribute when the two were associated.

Prior to constructing the bipartite graph, values from the source were normalized to eliminate inconsistencies based on cases, punctuations and storage format variations. Genres, categories, developers and publishers were formatted into normalized lists. Presence or absence features represented steam tags. Combining the identifier for a feature with the relation type resulted in identifiers that avoided duplicate entries due to identical values being entered using differing relation types. For example; tag::action or developer::valve.

Coverage rates were high for genres, categories, developers and publishers, while Steam tags had lower coverage at 81.17%. No inference was made concerning missing relationships. All edges within the bipartite graphs remain directly supported by the catalogue data.

3.6.2 Edge Weighting and Hyper-parameters

The various types of relations were assigned a base weight in order to express their respective influence on the degree of similarity between games. Tag relationships have the greatest weight as they contain much more detailed characteristics concerning the gameplay mechanics, themes and play style of each game. Genre and category relationships have less detail as they represent more general descriptions of games. The relationship between developers or publishers will be considered as additional indicators. Organisational similarity **does not** automatically mean that there will also be similarities with regards to the gameplay.

In table 1, the most important hyper-parameters for the graphs are shown. Because these values had been manually chosen (instead of being estimated using labelled relevance data), we consider them as model-specific choices and therefore do not treat them as "optimal" based on empirical results. We mainly justify the hierarchy of the values.

Table 1: Hyper-parameters for Graph Recommenders

Relation Type	Weight	Role
Tag	1.00	Fine-grained Gameplay Signal
Genre	0.80	Broad Game Classification
Category	0.60	Gameplay/Platform Relationship
Developer	0.40	Supplementary Production Signal
Publisher	0.30	Supplementary Organizational Signal

An Inverse Document Frequency (IDF) weighting has been added to avoid that high-frequent entities dominate the similarity measure. The weighting function of feature-entity f , can be expressed by:

$$\text{IDF}(f) = \log \left(\frac{N + 1}{df(f) + 1} \right) + 1 \quad (3.6)$$

where N is the total amount of games, and $df(f)$ represents the amount of connections that exist between this entity and other games. The final edge-weight of a connection between the two games g and f has been defined as:

$$w(g, f) = w_{\text{relation}}(f) \cdot \text{IDF}(f) \quad (3.7)$$

Thus, the weight of a specific relation-type and the uniqueness of the corresponding entity are separated. To illustrate this concept, assume that two tags have the same base-relation-weight, however the first tag is much rarer than the second tag. Thus, when calculating the similarity between two games, it makes sense to weigh a connection with the rare tag stronger than a connection with a very frequent tag.

Entities that are connected to exactly one game are deleted from the graph prior to computing the similarity measures. Since such singleton-nodes cannot contribute any evidence regarding the similarity of two different games. After deleting all these nodes, the number of remaining weighted connections in the graph decreased from 1,833,397 initial connections between games and features to 1,749,653 connections.

3.6.3 Representation of Sparse Graphs, Similarities, and Candidate Generation

After converting the weighted bipartite graph into a sparse game-feature incidence matrix containing 89,618 rows (games) and 25,873 column entries (features), after removing singletons, this resulted in 1,749,653 non-zero relations, which corresponded to about 0.075% density; 89,534 games retained at least one useful graph feature.

To eliminate the effect of vector magnitude on the degree of similarity that would occur solely due to the number of catalog-related attributes associated with each game vector, each game vector was normalized by the L2-norm.

Cosine similarity between query-game q and candidate-game x_i is then defined as:

$$S_{\text{graph}}(q, i) = q^T x_i \quad (3.8)$$

where both are already normalized by their L2-norm. Therefore, the dot-product (i.e., cosine similarity) represents weighted first-order shared relationships between the two games. This model also does NOT use any higher-order graph-traversal techniques (e.g., message-passing), nor learns graph-representations.

3.7 Hybrid Recommendation Framework

The final recommendation framework combines the outputs of the weighted metadata, graph-based and transformer semantic recommenders. These models were selected because they provide complementary recommendation signals: the metadata model captures explicit gameplay characteristics, the graph model captures structural relationships between games and catalogue entities, and the semantic model captures similarity in descriptive meaning.

The exploratory rich-text TF-IDF recommender was **not** included as an independent component of the final hybrid architecture. The final system therefore combines three ranked recommendation lists using Weighted Reciprocal Rank Fusion (RRF).

A rank-based fusion strategy was preferred to direct score aggregation because the similarity scores produced by the three source recommenders have different representations and interpretations. Combining their raw scores directly would therefore require additional calibration. RRF instead combines the relative rank assigned by each recommender.

3.7.1 Candidate Pool Construction

Each of the three source recommenders generated a ranked list of 50 candidates for every query game. The candidate tables retained the Steam application identifier, query identifier, game title, source-model rank and source-model similarity score.

Before fusion, the candidate files were validated to ensure that each model contained the same ten evaluation query games and exactly 50 candidates per query.

The three candidate lists were combined by `appid` using their union which allows games returned by any constituent recommender to participate in the Hybrid ranking.

3.7.2 Weighted Reciprocal Rank Fusion

The merged candidate lists were combined using **Weighted Reciprocal Rank Fusion**.

For candidate game i , the contribution from source recommender m was calculated as:

$$RRF_m(i) = \frac{w_m}{k + r_m(i)} \quad (3.9)$$

where w_m represents the weight assigned to recommender m , $r_m(i)$ represents the rank of candidate i within that recommender, and k is the RRF smoothing constant.

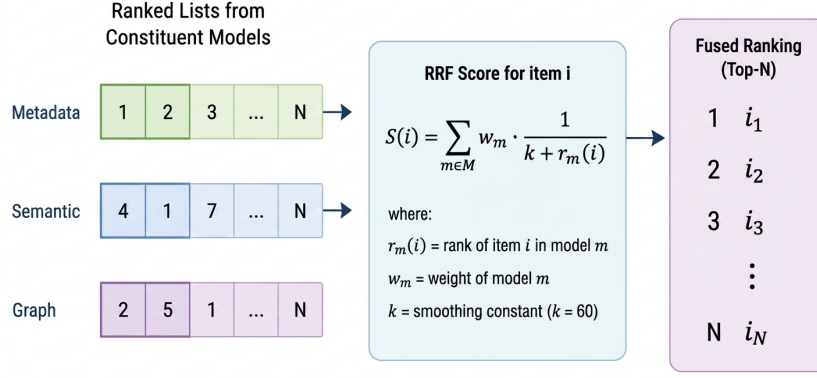


Figure 3: Weighted Reciprocal Rank Fusion Framework

The value $k = 60$ was adopted following Cormack et al. [7], who used this value as a fixed smoothing constant in the original RRF formulation. It was retained consistently across all fusion experiments in this project rather than being tuned on the evaluation labels. If a candidate was not returned by a particular source recommender, its contribution from that model was set to zero. The final hybrid score was then calculated as the sum of the available model contributions:

$$S_{\text{hybrid}}(i) = RRF_{\text{metadata}}(i) + RRF_{\text{graph}}(i) + RRF_{\text{semantic}}(i). \quad (3.10)$$

This formulation rewards candidates that appear near the top of one or more source rankings while avoiding direct comparison between the original similarity-score scales.

3.7.3 Model Weighting and Agreement

The three source recommenders were assigned different contributions to the hybrid ranking. The metadata and graph recommenders were each assigned a weight of 0.40, while the semantic recommender received a weight of 0.20:

$$w_{\text{metadata}} = 0.40, \quad w_{\text{graph}} = 0.40, \quad w_{\text{semantic}} = 0.20$$

These weights were treated as design assumptions rather than empirically optimal values. Their sensitivity to alternative configurations is examined in Chapter 4. The weighting gives greater influence to the structured metadata and graph signals while retaining semantic similarity as a complementary source of information. The weights sum to one and were applied consistently throughout formal evaluation and live recommendation.

In addition to the hybrid score, the system recorded a **model agreement** value for each candidate. This value represents the number of source recommenders in which the candidate appeared and therefore ranges from one to three. Model agreement was retained both for interpretability and as a secondary ranking criterion where candidates obtained the same hybrid RRF score.

3.7.4 Final Hybrid Ranking and Live Recommendation Interface

The candidates were first scored on their total Weighted RRF score, then on the model agreement (to break ties). For formal evaluations, the top 10 recommendations for each search were saved along with their model rankings, individual RRF scores, hybrid score and number of agreements. The outputs were checked to be sure there were 10 different recommended games per query, that no duplicate suggestions occurred, that no self-suggestions occurred, and that all 10 recommendations had been assigned a rank.

The last hybrid recommendation pipeline utilized a weighted RRF on the ranked results of the metadata, graph and semantic recommender models. Each of these models produced a candidate list of up to fifty candidates as input to the hybrid. Candidates were then sorted based upon the final hybrid score

and agreement between models. The Steam application identifier (**appid**) was retained for each of the recommended games such that the resultant rank list can be sent forward to the hardware requirement enrichment stage as described in section 3.8.

3.8 Hardware Requirement Enrichment Layer

A post-recommendation hardware enrichment layer was developed to attach PC system requirements to games produced by the Hybrid recommender. Rather than collecting hardware specifications for the complete Steam catalogue, requirements are retrieved only for recommended games, preserving the original Hybrid ranking while adding practical hardware information.

Steam application identifiers (**appid**) provide the link between the recommendation output and the hardware-requirement retrieval pipeline.

3.8.1 Integration and Dynamic Requirement Retrieval

Based upon the ranked output from the live Hybrid Recommender; For every game in the Hybrid Ranked Output, the appropriate Steam Application ID was fetched, and then used to retrieve the dynamic PC System Requirements via the **Steam API**. These two requirements (minimum & recommended) was stored where available. Since steam retrieves these in the form of a Semi-Structured HTML Text Field, all HTML tags was cleaned prior to additional processing. Only Requirements for games created by the Hybrid Recommender was pulled versus the full catalog of games. This allows for Hardware Information to be pulled on an incremental basis. The retrieval process therefore follows the sequence:

Hybrid Recommendation → Steam AppID → Steam Requirements → Hardware Processing

3.8.2 Requirement Parsing and Structured Representation

Steam system requirements are supplied as semi-structured text rather than as consistently formatted numerical attributes. A parsing stage was therefore implemented to convert the retrieved requirement descriptions into a more structured representation.

The parser searches the minimum and recommended requirement text independently for operating system, processor, memory, graphics and storage information. Alternative field labels are supported where possible. For example, processor information may appear under either **Processor** or **CPU**, while graphics information may appear under **Graphics**, **Video Card** or **GPU**.

CPU and GPU requirements are currently retained as textual descriptions because Steam specifications frequently contain multiple models, manufacturers and free-form qualification text. Memory and storage requirements are additionally converted into numerical gigabyte values where a recognised quantity can be identified. Values expressed in megabytes are converted into gigabytes during this process.

Minimum and recommended specifications are stored separately using structured fields such as:

- **min_cpu**, **min_ram_gb**, **min_gpu** and **min_storage_gb**;
- **rec_cpu**, **rec_ram_gb**, **rec_gpu** and **rec_storage_gb**.

Operating-system information is also retained for both requirement levels. The final hardware enrichment function combines these structured fields with the corresponding hybrid recommendation records, producing a single output containing recommendation relevance and game-specific hardware information.

At the current implementation stage, the hardware component performs requirement retrieval, parsing and enrichment of Hybrid recommendations, but does not compare these requirements with a user-supplied hardware profile. CPU and GPU requirements are retained as free-form text because Steam specifications are not consistently standardised for direct performance comparison. The implemented component is therefore treated as a hardware-requirement enrichment layer rather than a complete hardware-compatibility recommender.

Chapter 4

Critical Evaluation

4.1 Evaluation Design

In order to provide a comparison of the principal query-driven recommendation systems all were run under an identical experimental evaluation. The weighted Metadata, Transformer-based Semantic, Graph-based and Hybrid recommenders were evaluated based upon the same set of formal query games and the same Top-10 recommendation cutoff. Therefore the primary factors responsible for the variations seen in the results are due to variation in the models' behaviors and not variation in the input data used during testing.

The evaluation examined a number of distinct aspects of recommendation quality. The relevance of a given recommendation was determined using **Precision @ 10**, whereas the quality of a given ranking was determined through the use of Normalized Discounted Cumulative Gain at rank 10 (**NDCG@10**). The intra-list diversity of a given recommendation list was also evaluated using **Diversity @ 10**. Finally, the time taken by each approach to generate recommendations was compared using runtime metrics. In addition two supplemental analytical studies were completed. Initially the Jaccard Similarity metric was applied to assess the amount of intersection between the Metadata and Graph recommenders; this is because both models utilize the same structural catalogue metadata. Secondly, a sensitivity study was completed with respect to the Reciprocal Rank Fusion (RRF) weights to evaluate if the Hybrid recommender's behavior is sensitive to its initial weighting configuration.

For the purposes of formally evaluating the Metadata, Semantic and Graph recommenders each generated ranked candidate lists for the same query games. Additionally, the Hybrid recommender generated ranked candidate lists by fusing the ranks of the three other recommenders via Weighted Reciprocal Rank Fusion and selecting the top ten candidates. Lastly, Steam Application Identifiers were preserved across the entire pipeline, thus allowing for a consistent identifier to be utilized when correlating recommendations produced by each of the different models.

Since no explicit relevance labels for each query-candidate game pair existed within the provided Steam dataset, a secondary process was required to assign such relevance labels. Prior to assigning these labels, duplicate recommendations generated by each of the models for the same query were removed; hence each unique query-candidate pair would receive one label. Subsequently, each of the aforementioned labels were assigned to their respective model outputs; thereby assuring that any given candidate would have been evaluated identically regardless of which model returned it.

Therefore, the focus of the current evaluation is on comparative recommendation performance and not personalized preference prediction. As noted previously, this corresponds directly to the system developed wherein users enter a game title as a search query and receive relevant titles of games in response.

4.1.1 Formal Query Selection

A fixed set of ten games was used as the formal evaluation query set. The queries were selected to provide coverage across contrasting genres, gameplay styles and catalogue characteristics rather than concentrating the evaluation on a single type of game. The intention was to examine whether the

recommendation approaches behaved consistently across different forms of game content while keeping the evaluation set small enough to support detailed relevance assessment.

The selected evaluation queries are shown in Table 2.

Table 2: Formal Query Games used for Evaluation

No.	Query Game
1	Counter-Strike 2
2	Grand Theft Auto V Legacy
3	Stardew Valley
4	Sid Meier’s Civilization VI
5	Hollow Knight
6	The Witcher 3: Wild Hunt
7	Hades
8	Deep Rock Galactic
9	Factorio
10	Phasmophobia

The same ten queries were used across the Metadata, Semantic, Graph and Hybrid recommenders. Each model contributed ten final recommendations per query, producing 400 model-specific evaluation rows. However, the same candidate game could be returned by more than one recommender for the same query. After removing the duplicates from the query–candidate level, pool contained 246 unique pairs requiring relevance judgments.

The query set should **not** be interpreted as statistically representative of the complete Steam catalogue. Instead, it provides a controlled and varied evaluation sample that allows the behaviour of the different recommendation approaches to be compared under identical conditions.

4.1.2 Development and Evaluation Separation

A separate distinction was maintained between games used during development and those reserved for formal evaluation. During model construction, a small set of example queries was used for qualitative inspection, debugging and sanity checking. These development queries included titles such as *ELDEN RING*, *Red Dead Redemption 2*, *PUBG: BATTLEGROUNDS* and *Assassin’s Creed Odyssey*.

These games were **not** part of the ten-game formal evaluation set. The formal evaluation queries were therefore used only after the recommendation pipelines and model configurations had been established. This separation reduced the risk of adjusting model behaviour specifically around the examples later used for quantitative comparison.

4.1.3 Evaluation Methodology – Relevance Judgment Process

Since the Steam Catalogue, utilized in this research, contains no relevance labels (e.g., relevance judgments) that indicate which games are suitable recommendations for other games, an independent relevance judgment process had to be implemented to facilitate the quantitative evaluation of the various recommendation algorithms.

Each of the ten formal query games yielded top-ten results from the four types of recommendation models - metadata, semantic, graph and hybrid. Since the same candidate game may appear among the top-ten candidates from multiple models, duplicate query-candidate pairs were eliminated prior to the relevance judgment task. After removing duplicates, there existed 246 distinct query-candidate pairs among the 400 possible recommendation outcomes provided by each model. One of these unique pairs was evaluated once and subsequent evaluations for those pairs were based on the previously established relevance score for those specific pairs regardless of which model output they occurred in.

An ordinal relevance scale with three levels was utilized:

When performing the evaluation, all available information about each game including genre, steam tag, gameplay mechanics and overall similarity between the titles of the query and candidate games were taken into consideration. When a candidate strongly correlated with the query on the most critical aspects of

Table 3: Relevance Scale used for Query–Candidate Assessment

Score	Interpretation
0	Irrelevant or weakly related to the query game
1	Partially relevant, with some meaningful similarities
2	Highly relevant, with strong similarity in important game characteristics

both games, a score of 2 was assigned. When the relationship was somewhat plausible but indirect, a score of 1 was applied. A score of 0 was assigned when there existed little to no likely association.

The assessments of relevance judgments made use of the OpenAI’s **GPT-5.6 Sol** model. In assessing a unique query-candidate pair, an assessment would consider the game(s) genre, together with their respective Steam tag, and general gameplay styles, themes or settings, play modes, and overall player experience. When determining the relevance ratings of the query-candidate pairs, the assessment did NOT make use of any of the specific model-related data including but not limited to: the ranked recommendation produced by the recommender; the similarity score between the query and the candidate; model agreement on the candidate being recommended; nor the identification of the recommender model producing the candidate.

The ChatGPT model was instructed to produce a rating from one of three possible relevance ratings specified in Table 3. The judgments made by the LLM model were treated as a proxy for relevance instead of a reference to what users preferred. Although each judgment was labeled with a label that was assigned based on the judgment, no validation through an evaluation by a human user occurred. Therefore, the Precision@10 and NDCG@10 results presented within this dissertation are to be considered as comparisons of performance based on the structured assessment process of determining relevance, not as evidence directly related to how satisfied users were or how accurate the personalized preferences would have been.

4.2 Metrics Evaluation

Four different types of evaluation metrics were applied to evaluate the performance of the proposed recommendation techniques: **Precision@10**, Normalized Discounted Cumulative Gain at rank 10 (**NDCG@10**), **Intra-list Diversity@10**, and **runtime**. The combination of these four metrics allows us to evaluate how well the proposed recommendation approaches perform in terms of providing high-quality recommendations, recommending good-ranked lists, creating diverse lists of recommended items, and performing efficiently.

4.2.1 Precision@10

Precision@10 is a measure of what proportion of the ten highest rated recommendations are relevant. In previous section, the three levels of relevance (highly relevant, partially relevant, not relevant) were defined according to the relevance scores of 2, 1, and 0 respectively.

Precision@10 can be expressed as follows:

$$P@10 = \frac{1}{10} \sum_{i=1}^{10} \mathbb{I}(rel_i > 0) \quad (4.1)$$

where rel_i is the relevance score assigned to the item recommended at rank i , and I is an indicator function which equals 1 if the recommendation is relevant and 0 otherwise. Therefore, Precision@10 increases as more of the top 10 recommendations have some kind of relevance to the query game.

4.2.2 NDCG@10

As mentioned earlier, precision measures the number of correct answers obtained but it does not account for the order in which they appear. NDCG@10 was therefore deployed to evaluate ranking quality based

on the full graded relevance scale (highly relevant, partially relevant and not relevant) [12].

Discounted cumulative gain was calculated at rank 10 (DCG@10) as follows:

$$DCG@10 = \sum_{i=1}^{10} \frac{2^{rel_i} - 1}{\log_2(i + 1)} \quad (4.2)$$

Afterward, an ideal ranking was created by arranging in descending order the same relevance scores used previously. Then, computed the corresponding ideal discounted cumulative gain, IDCG@10, which corresponds to the best possible value for DCG that could be achieved for those recommendations. Finally, normalized discounted cumulative gain at rank 10, NDCG@10 is given by:

$$NDCG@10 = \frac{DCG@10}{IDCG@10} \quad (4.3)$$

Thus, NDCG@10 varies from zero to one and values close to one will denote highly relevant recommendations appearing first in the ranked list.

4.2.3 Intra-list Diversity@10

Another way to analyze the quality of recommendations is to look into diversity. To do so, available genres and Steam tag names as attributes for each recommended game were used. Also, Jaccard similarity as a measure to quantify the similarity of the two games a and b in terms of their attributes was implemented. The Jaccard similarity is defined as:

$$J(a, b) = \frac{|F_a \cap F_b|}{|F_a \cup F_b|} \quad (4.4)$$

where F_a and F_b are the attribute sets (genres and tags) of games a and b respectively. On the other hand, we define pairwise dissimilarity as:

$$D(a, b) = 1 - J(a, b) \quad (4.5)$$

Finally, intra-list diversity at rank 10 (ILD@10) is computed as the average pairwise dissimilarity among all distinct pairs of games in the Top-10 list. Thus, higher ILD@10 means that there exists much variation between the elements in the returned list whereas low values mean that most of the games share many common attributes in their catalogues.

4.2.4 Runtime

Runtime was recorded separately for each of the ten formal query games within the constituent recommender pipelines and stored in model-specific runtime files. The evaluation subsequently reported the mean per-query runtime for each constituent recommender.

For the Hybrid approach, end-to-end runtime for each query was calculated as the sum of the Metadata, Semantic and Graph recommendation runtimes and the additional RRF fusion runtime, before averaging across the ten evaluation queries.

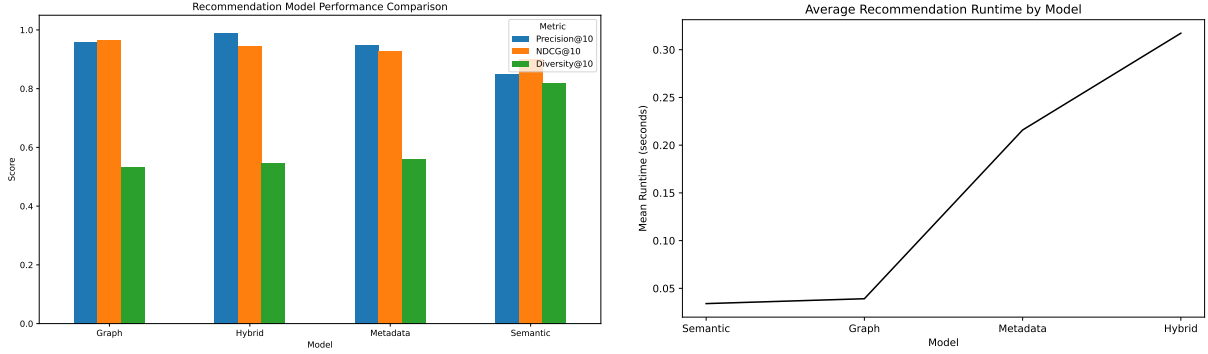
4.3 Comparison of Recommendation Approaches

The four principal recommendation approaches were compared using the evaluation metrics described in Section 4.2. Table 4 summarises the average Precision@10, NDCG@10, Intra-list Diversity@10 and mean runtime across the ten formal evaluation queries.

Figure 4 illustrates the contrasting behaviour of the four recommendation approaches across effectiveness, diversity and computational performance. Figure 4a shows that the Graph and Hybrid recommenders

Table 4: Overall Quantitative Comparison of the Recommendation Models

Model	Precision@10	NDCG@10	Diversity@10	Mean Runtime (s)
Graph	0.96	0.9638	0.5343	0.0391
Hybrid	0.99	0.9456	0.5462	0.3173
Metadata	0.95	0.9295	0.5610	0.2159
Semantic	0.85	0.9011	0.8187	0.0340



(a) Comparison of Precision@10, NDCG@10 and Diversity@10.

(b) Mean runtime comparison across the recommendation models.

Figure 4: Quantitative Comparison of Recommendation Approaches in terms of Effectiveness and Runtime

achieve stronger relevance-oriented performance, whereas the Semantic recommender produces substantially greater intra-list diversity. Figure 4b further shows that the Semantic and Graph models have the lowest mean runtimes, while the Hybrid recommender requires the highest computational cost due to the execution of all three constituent models followed by the fusion stage.

The results demonstrate that no single recommendation method achieves superior performance across all four evaluation criteria. Instead, each approach exhibits different strengths in terms of relevance, ranking quality, recommendation variety and computational cost.

4.3.1 Ranking Quality and Relevance

The Hybrid recommender exhibited the highest **Precision @10** value of **0.99**. Therefore, it follows from this result that nearly all games included in the Hybrid’s Top-10 lists for each query game were found to be at least partially related to them. A slightly lower Precision @10 of **0.96** was exhibited by the Graph recommender, whereas the Metadata model achieved a Precision @10 of **0.95**. The lowest Precision @10 of 0.85 was exhibited by the Semantic recommender.

However, when ordered according to **NDCG @10**, a different ordering was established. The Graph recommender achieved the highest value of **0.9638**, demonstrating that its highly relevant recommendations were typically positioned closer to the top of the ranked lists. The Hybrid recommender also achieved an NDCG @10 of **0.9456**, followed closely behind by the Metadata model at 0.9295 and then by the Semantic model at **0.9011**.

Therefore, these results indicate that the Hybrid recommender has successfully enhanced the general proportion of relevant recommendations; however, it has failed to surpass the Graph model in ranking quality. Therefore, the Graph model offered the highest observed NDCG@10 among the evaluated approaches, whereas the Hybrid model achieved the highest level of relevance rate overall for its Top-10 ranked recommendations.

4.3.2 Recommendation Diversity

Additionally, it is demonstrated through the results that the Semantic recommender has generated significantly greater levels of variety amongst its recommendation lists than the other methods examined;

thusly exhibiting a **Diversity @10** value of **0.8187**. These results suggest that the semantic recommender has selected recommendations with fewer commonalities amongst themselves in terms of both genre and tag-based characteristics. As such, the Semantic recommender has therefore created greater variability in terms of catalogue properties amongst its recommendations.

On the contrary, the remainder of the models have generated much lower and relatively comparable levels of diversity. Specifically, the Metadata model achieved a Diversity @10 value of **0.5610**, whereas the Hybrid model exhibited a Diversity @10 value of **0.5462**, and finally, the Graph model exhibited a Diversity @10 value of **0.5343**.

This decrease in diversity exhibited by both Metadata and Graph models is reflective of their reliance upon structured catalogue attributes including genres and tags as input into their respective recommendation processes. Consequently, since two of the three constituent recommenders utilized within the Hybrid model are similarly dependent upon structured catalogue relationships, it follows that the Hybrid model has inherited aspects of this characteristic as well.

As such, it is evident that the Semantic model presents a distinct trade-off. On one hand, it generates the largest quantity of variety in terms of diversity amongst its recommendation lists; yet, on the other hand, this increased variety is associated with decreased Precision @10 and NDCG @10 values relative to those of other principal methods.

4.3.3 Computational Performance

Finally, it is apparent from the results that both the Semantic and Graph recommenders were characterized by being the quickest methods; specifically they exhibited mean query processing times of **0.0340** and **0.0391** seconds respectively.

Conversely, Metadata required an average time of **0.2159** seconds per query to generate recommendations.

Additionally, the Hybrid recommender had the longest mean query processing time of approximately **0.3173** seconds; hence representing the total time taken by the entire Hybrid recommendation process. As noted previously this time encompasses both execution of the individual metadata recommender and semantic recommender and graph recommender as well as execution of the resultant RRF fusion stage.

Although a longer query processing time is incurred due to this combined execution process; it is still less than one second per evaluated query within our experimental setting.

However, due to potential variations in either implementation or hardware upon which the hybrid recommendation system may operate; we would caution against interpreting these measured performance values as absolute representations of guaranteed system performance measures.

4.4 Metadata-Graph Overlap Analysis

This section describes a study using Jaccard similarity to analyze the overlap between the Metadata and Graph recommender algorithms in terms of recommending games at Top-10, Top-20 and Top-50 cut-offs. For a given set of recommended game A and another set of recommended games B, the Jaccard Similarity (JS) can be computed as follows:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \quad (4.6)$$

A value of zero indicates no shared recommendations, while a value of one indicates identical recommendation sets. The analysis was performed independently for each of the ten formal evaluation queries and then summarised across queries.

Table 5 summarises the resulting overlap.

Table 5: Metadata-Graph Recommendation Overlap at Different Cut-offs

Cut-off	Mean Jaccard	Median Jaccard	Mean Common Candidates
Top-10	0.4049	0.4286	5.6
Top-20	0.4114	0.4550	11.2
Top-50	0.4042	0.4600	27.9

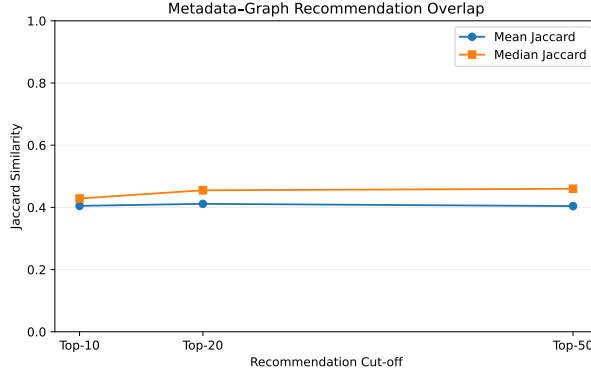


Figure 5: Metadata-Graph Recommendation Overlap

The mean Jaccard similarity remained close to 0.40 across all three recommendation cut-offs. At Top-10, the two models shared an average of 5.6 recommendations, increasing to 11.2 at Top-20 and 27.9 at Top-50. The relatively stable Jaccard values suggest that the overlap between the two models was not restricted only to the highest-ranked recommendations.

Given the consistent overlap observed among different cuts, it appears that the similarity between the two methods is not limited only to their top ranked recommendations. Nonetheless, since the Jaccard values remain under 1, conclude that these two methods do not produce identical candidate sets.

Therefore, results suggest a partial redundancy in contrast to a full duplication. As such, although both recommender systems utilize structured catalogue data, the Graph model returned some recommendations that were not returned by the Metadata model at the evaluated cut-offs.

4.5 RRF Weight Sensitivity Analysis

Weighted Reciprocal Rank Fusion (weighted RRF) was used by the Hybrid recommender to fuse the ranked output of the metadata, Graph and Semantic recommenders.

The weights used in the Hybrid recommender were initially set to **0.40, 0.40 and 0.20** for metadata, Graph and Semantic recommenders respectively.

These values were taken as design assumptions rather than parameter estimates derived from labelled relevant training data. To evaluate if the Hybrid recommender’s ability to generate recommendations is heavily reliant upon a particular weighting scheme, a sensitivity analysis of the Hybrid recommender with respect to different weighting schemes has been undertaken.

For the sensitivity analysis, the same formal evaluation queries, source candidate lists and rrf constant have been used but the model weights have been altered.

three additional weight configurations have been compared against the initial Hybrid recommender weight configuration. The four evaluated configurations are outlined below in table 6.

The results indicate that moderate changes to the RRF weights had very little effect on Hybrid recommendation performance. Precision@10 remained unchanged at 0.99 across all four configurations, while NDCG@10 varied only between 0.9425 and 0.9458.

The Metadata-heavy configuration achieved the highest NDCG@10 value of 0.9458; however, this represents an improvement of only 0.0002 over the original configuration. The Equal configuration also

Table 6: RRF Weight Sensitivity Results

configuration	metadata	graph	semantic	Precision@10	NDCG@10
Current	0.40	0.40	0.20	0.99	0.9456
Equal	0.33	0.33	0.33	0.99	0.9447
Metadata-heavy	0.50	0.30	0.20	0.99	0.9458
Graph-heavy	0.30	0.50	0.20	0.99	0.9425

produced a very similar NDCG@10 value of 0.9447, whereas the Graph-heavy setting produced the lowest value of 0.9425.

Following Figure shows that the performance differences between the tested configurations are small. The Hybrid recommender therefore appears relatively robust to moderate changes in the fusion weights.

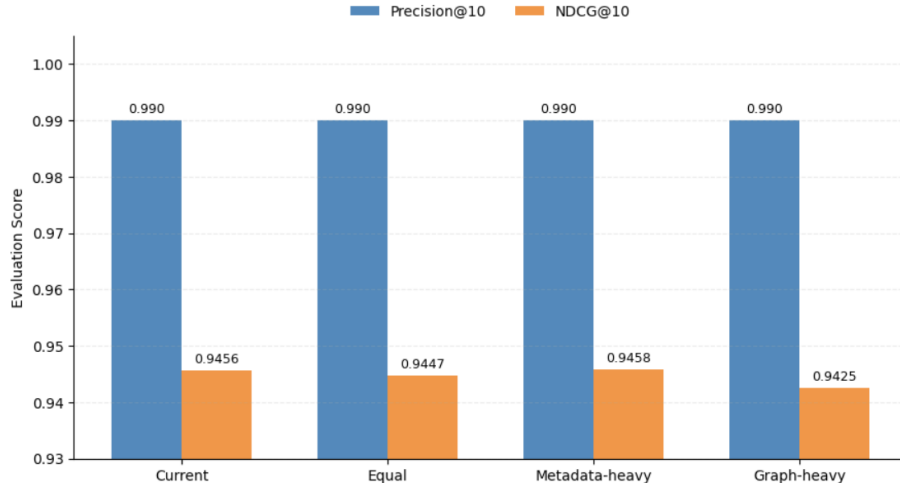


Figure 6: RRF Weight Sensitivity Analysis

Since the original 0.40/0.40/0.20 configuration achieved performance that was effectively comparable to the best tested alternative, it was retained for the final Hybrid recommender. The sensitivity analysis is therefore used to support the robustness of the original design assumption rather than to claim that the selected weights are globally optimal.

4.6 Qualitative Case Studies

Although, the quantitative analysis provides a general overview of the four recommendation methods that we tested, it does not allow us to see the detail of how the models behave with respect to individual recommendation lists. Therefore, in order to gain further understanding about how the models may behave differently in practice when they produce recommendations from a particular query (i.e., user search), two formal evaluation queries were evaluated qualitatively.

As a way to illustrate the differences in type and ordering of games retrieved using the Metadata, Graph, Semantic and hybrid recommenders.

Table 7 represents the top-3 recommendation for each of the recommended game lists (for *factorio* and *grand theft auto v legacy*) to provide insight into how the models may behave differently in practice. In this context, the examples provided represent an illustration of how the models may generate recommendation lists rather than providing an additional quantitative method to evaluate the quality of the models.

For *Factorio*, the four recommenders produced visibly different Top-3 lists. The Metadata recommender returned *Survival Vacancy*, *Techtonica* and *Satisfactory*, while the Graph recommender returned *Voxel Tycoon*, *Kubifaktorium* and *RAILGRADE*. The Semantic recommender produced another distinct set consisting of *Factor Y*, *Incremental Factory* and *Facteroids*. The Hybrid recommender ranked *Satisfactory* first, followed by *FOUNDRY* and *ReFactory*, illustrating how the fused ranking can differ from each individual constituent list.

Table 7: Selected Qualitative Recommendation Examples

<i>Factorio</i>		<i>Grand Theft Auto V Legacy</i>	
Model	Top-3 Recommendations	Model	Top-3 Recommendations
Metadata	1. Survival Vacancy 2. Techtonica 3. Satisfactory	Metadata	1. Grand Theft Auto V Enhanced 2. Grand Theft Auto IV: The Complete Edition 3. Red Dead Redemption 2
Graph	1. Voxel Tycoon 2. Kubifaktorium 3. RAILGRADE	Graph	1. Grand Theft Auto V Enhanced 2. Grand Theft Auto IV: The Complete Edition 3. Just Cause 3
Semantic	1. Factor Y 2. Incremental Factory 3. Facteroids	Semantic	1. Grand Theft Auto V Enhanced 2. Grand Theft Auto III – The Definitive Edition 3. Grand Theft Auto: Vice City – The Definitive Edition
Hybrid	1. Satisfactory 2. FOUNDRY 3. ReFactory	Hybrid	1. Grand Theft Auto V Enhanced 2. Red Dead Redemption 2 3. Grand Theft Auto: Vice City – The Definitive Edition

For *Grand Theft Auto V Legacy*, all three constituent recommenders ranked *Grand Theft Auto V Enhanced* first, and the Hybrid recommender retained it at rank one. Beyond this common result, the rankings diverged. Metadata included *Grand Theft Auto IV: The Complete Edition* and *Red Dead Redemption 2*, Graph included *Grand Theft Auto IV: The Complete Edition* and *Just Cause 3*, while Semantic returned two other Grand Theft Auto titles. The Hybrid Top-3 combined these signals by returning *Grand Theft Auto V Enhanced*, *Red Dead Redemption 2* and *Grand Theft Auto: Vice City – The Definitive Edition*.

These examples demonstrate that the constituent recommenders cannot be treated as being interchangeable. Each recommender’s ranking can show both similarities and large variances as a result of the different representation techniques employed by each recommender. Hence, the hybrid technique employs these ranking signals instead of solely employing a single recommender’s behavior. Hence, the qualitative assessments are complementary to, rather than replacing, the quantitative comparisons outlined in previous sections.

Chapter 5

Conclusion

5.1 Summary of Contributions

This study represented the design and evaluation of a hybrid video game recommender that uses metadata, TF-IDF (Term Frequency-Inverse Document Frequency), transformer models, graph-based catalogue relationships, as well as a non-personalized baseline to rank items in a large steam catalog. Similarity representations included weighted metadata, TF-IDF of rich-text summaries, transformer-based semantics, graph-based representations, and a non-personalized baseline. The Metadata, Semantic and Bipartite Graph recommenders were then fused using Weighted Reciprocal Rank Fusion (RRF) to create a ranked output that combines the benefits of each method without requiring recalibration of the individual similarity representation weights. Thus, the Hybrid Framework created an effective way of integrating both structure and unstructured data within the same ranked recommendation output.

A structured comparative evaluation methodology was also defined. The four principal evaluated approaches, namely Metadata, Semantic, Graph and Hybrid, were compared using a predefined set of common query games with respect to relevance, ranking quality, intra-list diversity and processing time. In addition, the similarity between Metadata and Graph Recommenders was analyzed; as well as how sensitive the Hybrid Ranking is to alternative RRF weight combinations.

Lastly, the Recommendation Pipeline was enhanced by adding an additional Hardware Requirement Enrichment Layer. This layer retrieves dynamic minimum and recommended hardware requirements directly from Steam's catalog; caches previous queries so repeated requests do not need to be sent again; and transforms semi-structured hardware description strings into structured CPU, GPU, Memory, Storage, and Operating System fields. Therefore, this layer adds relevant system requirements to each recommendation output while remaining distinct from the relevance-based ranking component. The hardware component was not included in the formal recommender evaluation because it does not currently perform user-specific compatibility assessment or hardware-based re-ranking.

5.2 Achievement of Project Objectives

The objectives of the project were successfully achieved in terms of the final implementation of the scope of this dissertation. The first objective focused on developing a large-scale Steam dataset that was compatible with query-driven recommendation. A catalogue of 89,618 games was audited, cleaned and preprocessed to produce a dataset that served as the common input source for all subsequent recommendation techniques and comparative evaluations.

The second objective was fulfilled through the development of several recommendation approaches that represent various ways in which games can be similar to each other. These include a Weighted Metadata Recommender; An Exploratory Rich-Text TF-IDF Recommender; A Transformer-Based Semantic Recommender; A Graph Recommender; And, Finally, A Non-Personalized Popularity Baseline.

The third objective was achieved by implementing a hybrid recommendation technique called the Weighted Reciprocal Rank Fusion (RRF) Hybrid. In order to implement the RRF Hybrid Recommendation Tech-

nique, the ranked outputs of the Metadata Recommender, Semantic Recommender and Graph Recommenders were fused together using RRF with weights assigned to each of the constituent ranking signals as follows: metadata = 0.40, semantic = 0.20, graph = 0.40. Thus, it was possible to combine the three ranking signals without needing to calibrate their underlying similarity scores on a common scale.

The fourth objective was met through a formal comparative evaluation of the primary Metadata, Semantic, Graph and Hybrid Recommenders. The evaluation assessed Precision @ 10, Normalized Discount Cumulative Gain (NDCG) @ 10, Intra-list diversity@10 and processing time over ten formally queried games. Additional analyses also evaluated metadata – graph based recommendation overlap, sensitivity of the hybrid recommender to alternate RRF weight configurations and qualitative differences between selected sets of recommendations.

The fifth objective was also achieved within the defined hardware-enrichment scope of the project. Through implementation of a Hardware Requirement Enrichment Pipeline, minimum and recommended pc system requirements were dynamically retrieved using the steam store api, cached locally and converted from semi-structured descriptions of requirements into structured information regarding processor, graphics, memory, storage and operating systems. These requirements were then associated with games recommended by the hybrid recommender. Aspects related to assessing compatibility of users' hardware and hardware-based re-ranking were not implemented and therefore represent potential areas for extending the system.

In addressing the research questions, **RQ1** was investigated through comparisons of the Metadata, Semantic and Graph Recommenders. Results indicated varying degrees of performance across individual approaches. For example, the Graph Recommender achieved the highest level of NDCG@10 (0.9638), while the semantic recommender generated the highest degree of intra-list diversity (0.8187). On the other hand, the metadata recommender generated precision @ 10 levels (0.95) and NCDG@10 levels (0.9295). Such results support the conclusion that the different game representations resulted in different recommendation behavior rather than producing equivalent rank ordered output.

In addressing **RQ2**, a direct comparison between the Hybrid Recommender and its constituent Metadata, Semantic and Graph Approaches was conducted. Although the Hybrid Recommender yielded higher precision @10 values (0.99) compared to its constituent recommenders (metadata = 0.95; semantic = 0.85; graph = 0.96), its NDCG@ 10 value (0.9456) was lower than that achieved by the graph recommender (0.9638). Therefore, such results demonstrate that no single hybrid model is better than all others for every criterion being evaluated; however, they do demonstrate that the combination of ranked outputs produces a different balance between relevance, ranking quality and diversity.

Finally, **RQ3** was investigated through the implementation of the Hardware Requirement Enrichment Pipeline. The system demonstrated that minimum and recommended pc hardware information can be obtained via dynamic retrieval using the steam store api, cached locally processed from semi-structured description into structured field information and added as an enrichment layer after generating hybrid recommendations.

5.3 Limitations

There are several factors which need to be taken into account when evaluating the results of this dissertation.

As stated previously, the formal evaluation used 10 game questions with varying genres, styles and catalog features. This provides a consistent framework for comparing models but these particular questions do not comprise a statistical representation of the total number of games available on Steam. Therefore, the results reported herein should be viewed as relative performance of the models evaluated against each other, based upon the specific subset of question presented during the evaluation.

Another limitation is that explicitly defined query-candidate relevance assessments were not obtained. Instead, the relevance assessments produced by the AI-assisted structured assessment process reflect comparative performance under the same structured proxy assessment procedure and cannot be directly interpreted as individualized user preference or satisfaction.

Both of the metadata and graph recommenders utilized similar structural catalogue data to generate recommendations including genres and Steam tags. The overlap analysis indicated that although there was some degree of overlap between the two recommendation sets, they both generated unique lists

of candidates. Thus, it appears that the graph recommender generated candidates that the metadata recommender did not include.

Lastly, the enrichment portion of the Hardware Requirements component currently only retrieves, caches, parses and presents minimum and recommended system requirements. Currently, it does not assess potential compatibility for users based on specific hardware nor performs any form of ranking based on user's hardware specifications. To accomplish either of these tasks, there needs to be an extension to the normalizing capabilities of the existing heterogeneous description for processors and/or graphics cards.

5.4 Future Work

There are several ways that this dissertation's system could be extended.

Firstly, there is much potential for extending the evaluation framework. Future studies might examine additional game query environments (with an increased number of games, including other major catalogues also), include evaluations via human relevance judgments or usability studies, and thus provide stronger evidence for the degree to which the recommendation frameworks generalize over all of the Steam catalog and to what extent the suggested titles match the actual interests of individual users.

Secondly, further research may extend the hybrid approach by systematically optimizing the combination method used within it. For example, instead of manually defining constant coefficients for the various elements of the combination process as was done here, future researchers may seek out automatically derived coefficients (data driven) or "learning to rank" strategies. In addition, the graph based recommender could be extended beyond considering the simple similarities in properties between entities to consider longer range connections, and other forms of graph traversal and/or learned graph encodings.

Thirdly, the most critical practical expansion concerns hardware compatibility. The current system will retrieve and structure both minimum and recommended computer system specifications; however, it does not currently evaluate those against user provided hardware profiles. Thus, future work could normalize processor and graphics card descriptions using benchmarks, hardware hierarchies, or performance tiers. Once normalized, recommendations may then be categorized according to whether they meet minimum or recommended compatibility criteria and could potentially serve as input to a secondary hardware aware re-ranking module.

Lastly, each of these expansions could be integrated into a more comprehensive interactive interface where users may pose queries about games, view explanation of their recommendations, and optionally supply their own hardware specifications.

5.5 Final Conclusion

This dissertation used a variety of ways to develop and evaluate an interactive hybrid query-driven video game recommendation system based on multiple aspects of similarity among games. The metadata, semantic and graph recommenders were shown to have advantages in terms of various metrics. The Hybrid recommender achieved the highest Precision@10 (0.99), while the Graph recommender achieved the highest observed NDCG@10 (0.9638). The Semantic recommender produced the highest intra-list diversity (0.8187).

The weighted reciprocal rank fusion provided a way to combine three methods that are complementary but would require direct comparison of their similarities if they were directly combined. The system was further expanded through the addition of a Hardware Requirement Enrichment layer that can retrieve, cache and structure the minimum and suggested specs for PCs. User specific hardware compatible assessments and hardware-based re-rankings are still outside the systems' capabilities.

In total, this research shows that combining complimentary ways to represent a catalog in a query-driven video game recommender has significant benefits, yet there are many areas where it could benefit from additional testing, assessing how relevant it is to humans, and developing new methods that take into account hardware.

Bibliography

- [1] Gediminas Adomavicius, Bamshad Mobasher, Francesco Ricci, and Alexander Tuzhilin. Context-aware recommender systems. *AI Magazine*, 32(3):67–80, 2011.
- [2] Vineeta Anand and Ashish Kumar Maurya. A survey on recommender systems using graph neural network. *ACM Transactions on Information Systems*, 43(1):1–49, 2024.
- [3] Michał Bałchanowski and Urszula Boryczka. A comparative study of rank aggregation methods in recommendation systems. *Entropy*, 25(1):132, 2023.
- [4] Robin Burke. Hybrid recommender systems: Survey and experiments. *User Modeling and User-Adapted Interaction*, 12(4):331–370, 2002.
- [5] Erion Çano and Maurizio Morisio. Hybrid recommender systems: A systematic literature review. *Intelligent Data Analysis*, 21(6):1487–1524, 2017.
- [6] Germán Cheuque, José Guzmán, and Denis Parra. Recommender systems for online video game platforms: The case of steam. In *Companion Proceedings of The 2019 World Wide Web Conference, WWW '19 Companion*, pages 763–771. Association for Computing Machinery, 2019.
- [7] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR '09*, pages 758–759. Association for Computing Machinery, 2009.
- [8] Andraž De Luisa, Jan Hartman, David Nabergoj, Samo Pahor, Marko Rus, Bozhidar Stevanoski, Jure Demšar, and Erik Štrumbelj. Predicting the popularity of games on steam. *CoRR*, abs/2110.02896, 2021.
- [9] Artemiy Ermilov et al. Steam games dataset 2025. Kaggle, 2025. Accessed: 4 September 2026.
- [10] Tianyu Gao, Xingcheng Yao, and Danqi Chen. Simcse: Simple contrastive learning of sentence embeddings. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 6894–6910. Association for Computational Linguistics, 2021.
- [11] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, et al. Array programming with numpy. *Nature*, 585:357–362, 2020.
- [12] Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of ir techniques. *ACM Transactions on Information Systems*, 20(4), 2002.
- [13] Hyeyoung Ko, Suyeon Lee, Yoonseo Park, and Anna Choi. A survey of recommendation systems: Recommendation models, techniques, and application fields. *Electronics*, 11(1):141, 2022.
- [14] Xiping Li, Jianghong Ma, Kangzhe Liu, Shanshan Feng, Haijun Zhang, and Yutong Wang. Category-based and popularity-guided video game recommendation: A balance-oriented framework. In *Proceedings of the ACM Web Conference 2024, WWW '24*, pages 3734–3744. Association for Computing Machinery, 2024.
- [15] Wes McKinney. Data structures for statistical computing in python. In *Proceedings of the 9th Python in Science Conference*, 2010.

- [16] John Nickolls, Ian Buck, Michael Garland, and Kevin Skadron. Scalable parallel programming with cuda. *ACM Queue*, 6(2):40–53, 2008.
- [17] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, et al. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [18] Gottfried Christophorus Prasetyadi and Mahfudin Mahfudin. Web-based expert system application to recommend computer specifications for gaming using backward chaining inference method. *Jurnal Sistem Informasi*, 13(2):110–117, 2017.
- [19] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, EMNLP-IJCNLP, pages 3982–3992, Hong Kong, China, 2019. Association for Computational Linguistics.
- [20] James Owen Ryan, Eric Kaltman, Timothy Hong, Michael Mateas, and Noah Wardrip-Fruin. People tend to like related games. In *Proceedings of the 10th International Conference on the Foundations of Digital Games*, FDG 2015, Pacific Grove, California, USA, 2015.
- [21] Gerard Salton and Christopher Buckley. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5):513–523, 1988.
- [22] Kaitao Song, Xu Tan, Tao Qin, Jianfeng Lu, and Tie-Yan Liu. Mpnet: Masked and permuted pre-training for language understanding. In *Advances in Neural Information Processing Systems*, volume 33, 2020.
- [23] Farman Ullah, Ghulam Sarwar, and Sungchang Lee. Social network and device aware personalized content recommendation. *Procedia Technology*, 17:528–533, 2014.
- [24] Vojtěch Vančura, Pavel Kordík, and Milan Straka. beformer: Bridging the gap between semantic and interaction similarity in recommender systems. In *Proceedings of the 18th ACM Conference on Recommender Systems*, RecSys ’24, pages 1102–1107. Association for Computing Machinery, 2024.
- [25] Markus Viljanen, Jukka Vahlo, Aki Koponen, and Tapio Pahikkala. Content based player and game interaction model for game recommendation in the cold start setting. *CoRR*, abs/2009.08947, 2020.
- [26] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, et al. Scipy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, 17:261–272, 2020.
- [27] Xiang Wang, Xiangnan He, Yixin Cao, Meng Liu, and Tat-Seng Chua. Kgat: Knowledge graph attention network for recommendation. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD ’19, pages 950–958. Association for Computing Machinery, 2019.
- [28] Eva Zangerle and Christine Bauer. Evaluating recommender systems: Survey and framework. *ACM Computing Surveys*, 55(8):1–38, 2022.
- [29] Jingmao Zhang, Zhiting Zhao, Yunqi Lin, Jianghong Ma, Tianjun Wei, Haijun Zhang, and Xiaofeng Zhang. Dual-phase playtime-guided recommendation: Interest intensity exploration and multimodal random walks. In *Proceedings of the 33rd ACM International Conference on Multimedia*, MM ’25, pages 6232–6241. Association for Computing Machinery, 2025.